
UART DESIGN USING SYSTEMVERILOG

A Digital Design and Verification Project

Language	SystemVerilog (IEEE 1800)
Protocol	UART - 8N1 Frame Format
Baud Rate	115200 bps
System Clock	100 MHz
Simulation Tool	Xilinx Vivado Simulator
Target FPGA	Digilent Basys3 (Artix-7)

Table of Contents

1. Summary (Quick Overview)
2. Introduction
3. Block Diagram / RTL Schematic
4. Design Code - SystemVerilog Interface (uart_if.sv)
5. Design Code - Top UART Wrapper (uart.sv)
6. Design Code - UART Transmitter (uart_tx.sv)
7. Design Code - UART Receiver (uart_rx.sv)
8. Top Module - FPGA Wrapper (top.sv)
9. Constraints File (uart.xdc)
10. Testbench (uart_tb.sv)
11. Simulation Results (Vivado)
12. FPGA Implementation - Basys3 Board
13. Conclusion

1. Summary (Quick Overview)

This project implements a UART (Universal Asynchronous Receiver Transmitter) module entirely in SystemVerilog. A UART is a simple communication protocol used to send data, one bit at a time, between two devices using only two wires (TX and RX) - no shared clock is needed between them.

In simple terms: the FPGA board reads 8 switch values (SW0-SW7) and continuously transmits that 8-bit value over the UART TX line to a PC. The PC receives this data through the USB-UART bridge on the Basys3 board, which can be viewed using a serial terminal (like PuTTY or Tera Term).

The design is broken into small, reusable modules:

- `uart_tx.sv` - converts parallel 8-bit data into a serial bit stream (Transmitter)
- `uart_rx.sv` - converts an incoming serial bit stream back into 8-bit parallel data (Receiver), with noise filtering
- `uart_if.sv` - a SystemVerilog interface that connects TX and RX modules cleanly using modports
- `uart.sv` - top-level UART wrapper combining TX and RX
- `top.sv` - the FPGA top module connecting switches, buttons, clock, and UART pins
- `uart.xdc` - the constraints file that maps the design to physical pins on the Basys3 board

The project was first simulated in Vivado to verify correct transmit/receive behaviour, and then synthesized, implemented, and programmed onto the Basys3 FPGA board for real hardware testing.

2. Introduction

UART (Universal Asynchronous Receiver Transmitter) is one of the oldest and most widely used serial communication protocols in digital systems. It is called 'asynchronous' because the transmitter and receiver do not share a common clock signal - instead, both sides agree on a fixed baud rate (bits per second) in advance, and timing is generated locally on each side.

A standard UART frame consists of:

- 1 Start bit (logic 0) - signals the beginning of a data frame
- 8 Data bits - the actual byte being transmitted (LSB first)
- 1 Stop bit (logic 1) - signals the end of the frame, line returns to idle

When the line is idle (no data being sent), it is held at logic 1. A transition from 1 to 0 indicates the start of a new byte.

2.1 Objective

- Design a fully functional UART transmitter and receiver in SystemVerilog
- Use SystemVerilog interfaces (uart_if) to make the design modular and readable
- Verify the design through simulation in Vivado
- Implement and test the design on a real Basys3 FPGA board

2.2 Tools and Hardware Used

Hardware Description Language	SystemVerilog
Simulation / Synthesis Tool	Xilinx Vivado
FPGA Board	Digilent Basys3 (Xilinx Artix-7)
Board Clock Frequency	100 MHz
Baud Rate	115200 bps
Data Width	8 bits
Communication Interface	USB-UART Bridge (PC <-> FPGA)

3. Block Diagram / RTL Schematic

The figure below shows the RTL schematic generated by Vivado after synthesis. It shows how the UART module connects to the input clock (CLK100MHZ), the reset button (BTNC), the incoming serial line (uart_rx), and the outgoing serial line (uart_tx). The internal registers (data_reg and valid_reg) hold the data to be transmitted and a flag indicating that new data is ready.

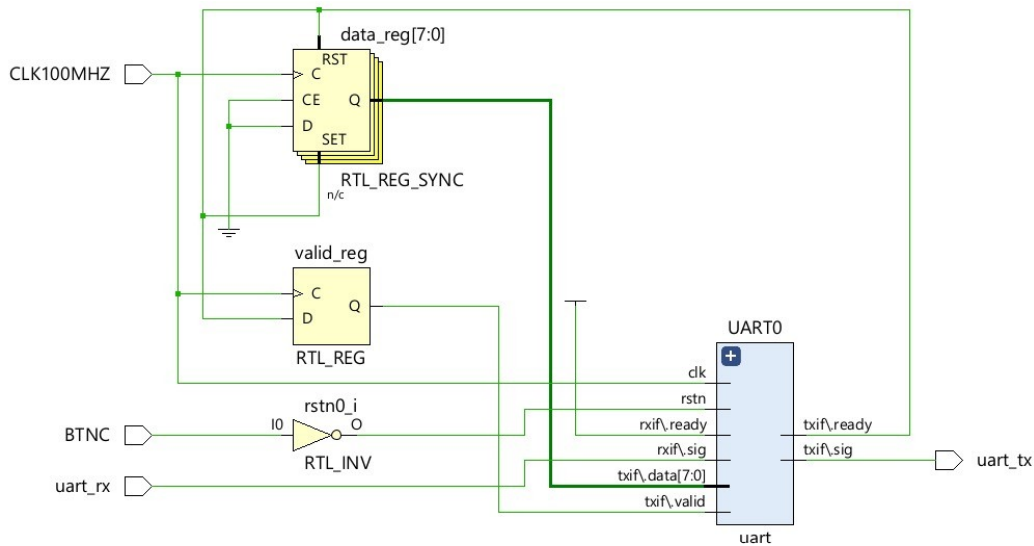


Figure 1: RTL Schematic of the UART top module (Vivado)

3.1 How Data Flows

- The 8 switches (SW[7:0]) on the Basys3 board are read every clock cycle
- This value is loaded into the UART transmitter (uart_tx) whenever it is ready (txif.ready = 1)
- uart_tx serializes the byte: sends start bit -> 8 data bits (LSB first) -> stop bit, at 115200 baud
- The serialized bit stream goes out on the uart_tx pin, through the USB-UART bridge, to the PC
- Similarly, uart_rx can receive a serial bit stream on uart_rx, filter out noise using a 5-sample majority filter, and reconstruct the 8-bit byte

4. Design Code - SystemVerilog Interface (uart_if.sv)

Instead of connecting TX and RX modules with many separate wires, this design uses a SystemVerilog interface. An interface bundles related signals (sig, data, valid, ready) into one connection, and 'modports' define which signals are inputs/outputs for the TX side versus the RX side. This keeps the code clean and avoids wiring mistakes.

```
interface uart_if
  #(parameter
    DATA_WIDTH = 8);

  logic          sig;
  logic [DATA_WIDTH-1:0] data;
  logic          valid;
  logic          ready;

  modport tx(output sig,
             input data,
             input valid,
             output ready);

  modport rx(input sig,
             output data,
             output valid,
             input ready);

endinterface
```

5. Design Code - Top UART Wrapper (uart.sv)

This module simply instantiates the transmitter and receiver sub-modules and connects them to the two interface instances (rxif and txif).

```
module uart
  #(parameter
    DATA_WIDTH = 8,
    BAUD_RATE   = 115200,
    CLK_FREQ    = 100_000_000)
  (uart_if.rx   rxif,
   uart_if.tx   txif,
   input logic  clk,
   input logic  rstn);

  uart_tx #(DATA_WIDTH, BAUD_RATE, CLK_FREQ)
  uart_tx_inst(
    .txif(txif),
    .clk(clk),
    .rstn(rstn)
  );

  uart_rx #(DATA_WIDTH, BAUD_RATE, CLK_FREQ)
  uart_rx_inst(
    .rxif(rxif),
    .clk(clk),
    .rstn(rstn)
  );
endmodule
```

6. Design Code - UART Transmitter (uart_tx.sv)

The transmitter is built as a 3-state Finite State Machine (FSM): STT_WAIT, STT_DATA, and STT_STOP.

- STT_WAIT: idle state. Line is held high (1). Waits until txif.valid = 1, then sends the start bit (0) and moves to STT_DATA
- STT_DATA: sends each of the 8 data bits one at a time (LSB first), waiting PULSE_WIDTH clock cycles between each bit
- STT_STOP: sends the stop bit (1) and returns to STT_WAIT, ready for the next byte

$PULSE_WIDTH = CLK_FREQ / BAUD_RATE = 100,000,000 / 115200 \sim 868$ clock cycles per UART bit.

```
module uart_tx
#(parameter
    DATA_WIDTH = 8,
    BAUD_RATE   = 115200,
    CLK_FREQ    = 100_000_000,

    localparam
        LB_DATA_WIDTH   = $clog2(DATA_WIDTH),
        PULSE_WIDTH     = CLK_FREQ / BAUD_RATE,
        LB_PULSE_WIDTH  = $clog2(PULSE_WIDTH),
        HALF_PULSE_WIDTH = PULSE_WIDTH / 2)
(uart_if.tx txif,
input logic clk,
input logic rstn);

typedef enum logic [1:0] {STT_DATA,
                        STT_STOP,
                        STT_WAIT
                        } statetype;

statetype          state;

logic [DATA_WIDTH-1:0] data_r;
logic                 sig_r;
logic                 ready_r;
logic                 data_cnt;
logic [LB_PULSE_WIDTH:0] clk_cnt;

always_ff @(posedge clk) begin
    if(!rstn) begin
        state    <= STT_WAIT;
        sig_r    <= 1;
        data_r    <= 0;
        ready_r   <= 1;
        data_cnt  <= 0;
        clk_cnt   <= 0;
    end
    else begin
        case(state)

            // STT_DATA: serialize and transmit data bits
            STT_DATA: begin
                if(0 < clk_cnt) begin
                    clk_cnt <= clk_cnt - 1;
                end
                else begin
                    sig_r <= data_r[data_cnt];
                    clk_cnt <= PULSE_WIDTH;

                    if(data_cnt == DATA_WIDTH - 1) begin
                        state <= STT_STOP;
                    end
                    else begin
                        data_cnt <= data_cnt + 1;
                    end
                end
            end
        endcase
    end
end
```

```

        end
    end

    // STT_STOP: assert stop bit
    STT_STOP: begin
        if(0 < clk_cnt) begin
            clk_cnt <= clk_cnt - 1;
        end
        else begin
            state <= STT_WAIT;
            sig_r <= 1;
            clk_cnt <= PULSE_WIDTH + HALF_PULSE_WIDTH;
        end
    end
end

// STT_WAIT: watch valid signal, assert start bit
STT_WAIT: begin
    if(0 < clk_cnt) begin
        clk_cnt <= clk_cnt - 1;
    end
    else if(!ready_r) begin
        ready_r <= 1;
    end
    else if(txif.valid) begin
        state <= STT_DATA;
        sig_r <= 0;
        data_r <= txif.data;
        ready_r <= 0;
        data_cnt <= 0;
        clk_cnt <= PULSE_WIDTH;
    end
end

default: begin
    state <= STT_WAIT;
end
endcase
end
end

assign txif.sig = sig_r;
assign txif.ready = ready_r;

endmodule

```

7. Design Code - UART Receiver (uart_rx.sv)

The receiver also uses a 3-state FSM (STT_WAIT, STT_DATA, STT_STOP), mirroring the transmitter, but in reverse - it deserializes incoming bits into a byte. It additionally includes a noise filter (majority5 function) which samples the incoming line 5 times and outputs the majority value (0 or 1), preventing glitches from corrupting data.

- STT_WAIT: watches the line for a falling edge (start bit). When detected, moves to STT_DATA
- STT_DATA: samples the line at the middle of each bit period and shifts it into data_tmp_r (LSB first)
- STT_STOP: waits for the stop bit (1) to confirm the frame ended correctly, then returns to STT_WAIT
- Once rx_done is asserted, the received byte is latched into rxif.data and rxif.valid is set high until the receiver (e.g., a display module) sets rxif.ready

```
module uart_rx
#(parameter
    DATA_WIDTH = 8,
    BAUD_RATE   = 115200,
    CLK_FREQ    = 100_000_000,

    localparam
        LB_DATA_WIDTH   = $clog2(DATA_WIDTH),
        PULSE_WIDTH     = CLK_FREQ / BAUD_RATE,
        LB_PULSE_WIDTH  = $clog2(PULSE_WIDTH),
        HALF_PULSE_WIDTH = PULSE_WIDTH / 2)
(uart_if.rx rxif,
 input logic clk,
 input logic rstn);

// noise removing filter - majority vote of 5 samples
function majority5(input [4:0] val);
    case(val)
        5'b00000, 5'b00001, 5'b00010, 5'b00100,
        5'b01000, 5'b10000, 5'b00011, 5'b00101,
        5'b01001, 5'b10001, 5'b00110, 5'b01010,
        5'b10010, 5'b01100, 5'b10100, 5'b11000:
            majority5 = 0;
        default: majority5 = 1;
    endcase
endfunction

logic [1:0] sampling_cnt;
logic [4:0] sig_q;
logic       sig_r;

always_ff @(posedge clk) begin
    if(!rstn) begin
        sampling_cnt <= 0;
        sig_q        <= 5'b11111;
        sig_r        <= 1;
    end
    else begin
        if(sampling_cnt == 0) begin
            sig_q <= {rxif.sig, sig_q[4:1]};
        end
        sig_r        <= majority5(sig_q);
        sampling_cnt <= sampling_cnt + 1;
    end
end

typedef enum logic [1:0] {STT_DATA,
                        STT_STOP,
                        STT_WAIT
} statetype;
```

```

statetype          state;

logic [DATA_WIDTH-1:0] data_tmp_r;
logic [LB_DATA_WIDTH:0] data_cnt;
logic [LB_PULSE_WIDTH:0] clk_cnt;
logic              rx_done;

always_ff @(posedge clk) begin
    if(!rstn) begin
        state      <= STT_WAIT;
        data_tmp_r <= 0;
        data_cnt   <= 0;
        clk_cnt    <= 0;
    end
    else begin
        case(state)

            // STT_DATA: deserialize and receive data
            STT_DATA: begin
                if(0 < clk_cnt) begin
                    clk_cnt <= clk_cnt - 1;
                end
                else begin
                    data_tmp_r <= {sig_r, data_tmp_r[DATA_WIDTH-1:1]};
                    clk_cnt    <= PULSE_WIDTH;

                    if(data_cnt == DATA_WIDTH - 1) begin
                        state <= STT_STOP;
                    end
                    else begin
                        data_cnt <= data_cnt + 1;
                    end
                end
            end

            // STT_STOP: watch stop bit
            STT_STOP: begin
                if(0 < clk_cnt) begin
                    clk_cnt <= clk_cnt - 1;
                end
                else if(sig_r) begin
                    state <= STT_WAIT;
                end
            end

            // STT_WAIT: watch start bit
            STT_WAIT: begin
                if(sig_r == 0) begin
                    clk_cnt <= PULSE_WIDTH + HALF_PULSE_WIDTH;
                    data_cnt <= 0;
                    state   <= STT_DATA;
                end
            end

            default: begin
                state <= STT_WAIT;
            end
        endcase
    end
end

assign rx_done = (state == STT_STOP) && (clk_cnt == 0);

logic [DATA_WIDTH-1:0] data_r;
logic                valid_r;

always_ff @(posedge clk) begin
    if(!rstn) begin
        data_r <= 0;
        valid_r <= 0;
    end
    else if(rx_done && !valid_r) begin

```

```
        valid_r <= 1;
        data_r  <= data_tmp_r;
    end
    else if(valid_r && rxif.ready) begin
        valid_r <= 0;
    end
end

assign rxif.data  = data_r;
assign rxif.valid = valid_r;
endmodule
```

8. Top Module - FPGA Wrapper (top.sv)

This is the top-level module that gets implemented on the Basys3 board. It connects the board's physical pins (clock, button, switches, UART pins) to the uart module.

- CLK100MHZ - the board's 100 MHz onboard clock
- BTNC - the center push-button, used as an active-low reset (rstn = ~BTNC)
- SW[7:0] - 8 slide switches, providing the data byte to transmit
- uart_rx / uart_tx - the physical serial lines connected to the USB-UART bridge
- On every clock edge, if the transmitter is ready (txif.ready), the current switch values are loaded and txif.valid is asserted, triggering a new transmission

```
`include "uart_if.sv"
module top(
    input  logic CLK100MHZ,
    input  logic BTNC,

    input  logic [7:0] SW,

    input  logic uart_rx,
    output logic uart_tx
);

    uart_if rxif();
    uart_if txif();

    uart UART0(
        .rxif(rxif),
        .txif(txif),
        .clk(CLK100MHZ),
        .rstn(~BTNC)
    );

    assign rxif.sig = uart_rx;
    assign uart_tx = txif.sig;

    always_ff @(posedge CLK100MHZ) begin
        if(txif.ready) begin
            txif.data <= SW;
            txif.valid <= 1'b1;
        end
        else begin
            txif.valid <= 1'b0;
        end
    end

    assign rxif.ready = 1'b1;

endmodule
```

9. Constraints File (uart.xdc)

The .xdc (Xilinx Design Constraints) file maps the logical port names used in top.sv to the actual physical pins on the Basys3 FPGA chip. Without this file, Vivado would not know which physical pin corresponds to which signal.

```
## Clock
set_property PACKAGE_PIN W5 [get_ports CLK100MHZ]
set_property IOSTANDARD LVCMOS33 [get_ports CLK100MHZ]
create_clock -add -name sys_clk_pin -period 10.000 -waveform {0 5} [get_ports CLK100MHZ]

## Center Button
set_property PACKAGE_PIN U18 [get_ports BTNC]
set_property IOSTANDARD LVCMOS33 [get_ports BTNC]

## USB-UART Bridge
## PC --> FPGA
set_property PACKAGE_PIN B18 [get_ports uart_rx]
set_property IOSTANDARD LVCMOS33 [get_ports uart_rx]

## FPGA --> PC
set_property PACKAGE_PIN A18 [get_ports uart_tx]
set_property IOSTANDARD LVCMOS33 [get_ports uart_tx]

## Switches
set_property PACKAGE_PIN V17 [get_ports {SW[0]}]
set_property PACKAGE_PIN V16 [get_ports {SW[1]}]
set_property PACKAGE_PIN W16 [get_ports {SW[2]}]
set_property PACKAGE_PIN W17 [get_ports {SW[3]}]
set_property PACKAGE_PIN W15 [get_ports {SW[4]}]
set_property PACKAGE_PIN V15 [get_ports {SW[5]}]
set_property PACKAGE_PIN W14 [get_ports {SW[6]}]
set_property PACKAGE_PIN W13 [get_ports {SW[7]}]

set_property IOSTANDARD LVCMOS33 [get_ports {SW[*]}]
```

10. Testbench (uart_tb.sv)

A self-checking testbench was written to verify the uart module before moving to hardware. The testbench instantiates the DUT (Device Under Test), generates a 100 MHz clock, and connects the transmitter output directly to the receiver input (rxif.sig = txif.sig) - this is called loopback testing. Whatever byte is transmitted should come back out of the receiver unchanged.

10.1 How the Testbench Works

- A 100 MHz clock is generated using 'forever #5 clk = ~clk' (10 ns period)
- The DUT is reset (rstn = 0) for 20 clock cycles, then released (rstn = 1)
- The receiver is configured to always be ready (rxif.ready = 1'b1)
- The send_byte task waits until the transmitter is ready (txif.ready), then loads txif.data and pulses txif.valid for one clock cycle
- Four test bytes are sent one after another: 0x55, 0xA5, 0xF0, and 0x3C
- After each byte, the testbench waits for rxif.valid and compares rxif.data against the expected value, printing PASS or FAIL
- 0x55 (01010101) and 0xA5 (10100101) are classic alternating-bit patterns used to check that every bit position toggles correctly

```

`timescale 1ns/1ps

`include "uart_if.sv"

module uart_tb;

    parameter DATA_WIDTH = 8;
    parameter BAUD_RATE   = 115200;
    parameter CLK_FREQ    = 100_000_000;

    logic clk;
    logic rstn;

    uart_if rxif();
    uart_if txif();

    // DUT
    uart #(
        .DATA_WIDTH(DATA_WIDTH),
        .BAUD_RATE(BAUD_RATE),
        .CLK_FREQ(CLK_FREQ)
    ) DUT (
        .rxif(rxif),
        .txif(txif),
        .clk(clk),
        .rstn(rstn)
    );

    // 100 MHz Clock
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Loopback Connection
    assign rxif.sig = txif.sig;

    // Receiver always ready
    assign rxif.ready = 1'b1;

    // Task to transmit one byte
    task send_byte(input [7:0] data);
    begin
        @(posedge clk);

        wait(txif.ready == 1'b1);

        txif.data <= data;
        txif.valid <= 1'b1;

        @(posedge clk);
        txif.valid <= 1'b0;

        $display("[%0t] TX Sent = %h", $time, data);
    end
    endtask

    // Main Test Sequence
    initial begin

        // Initialize
        txif.data = 8'h00;
        txif.valid = 1'b0;

        rstn = 0;

        repeat(20) @(posedge clk);

        rstn = 1;

        $display("UART TEST STARTED");

        // Test Case 1

```

```

send_byte(8'h55);
wait(rxif.valid);
if(rxif.data == 8'h55)
    $display("PASS : Received = %h", rxif.data);
else
    $display("FAIL : Expected=55 Received=%h", rxif.data);

// Test Case 2
send_byte(8'hA5);
wait(rxif.valid);
if(rxif.data == 8'hA5)
    $display("PASS : Received = %h", rxif.data);
else
    $display("FAIL : Expected=A5 Received=%h", rxif.data);

// Test Case 3
send_byte(8'hF0);
wait(rxif.valid);
if(rxif.data == 8'hF0)
    $display("PASS : Received = %h", rxif.data);
else
    $display("FAIL : Expected=F0 Received=%h", rxif.data);

// Test Case 4
send_byte(8'h3C);
wait(rxif.valid);
if(rxif.data == 8'h3C)
    $display("PASS : Received = %h", rxif.data);
else
    $display("FAIL : Expected=3C Received=%h", rxif.data);

$display("UART TEST COMPLETED");

#10000;
$finish;
end

```

endmodule

11. Simulation Results (Vivado)

The testbench was run on the Vivado behavioural simulator. The waveform below shows the four test bytes (0x55, 0xA5, 0xF0, 0x3C) being transmitted through txif and correctly received back through rxif via the loopback connection.

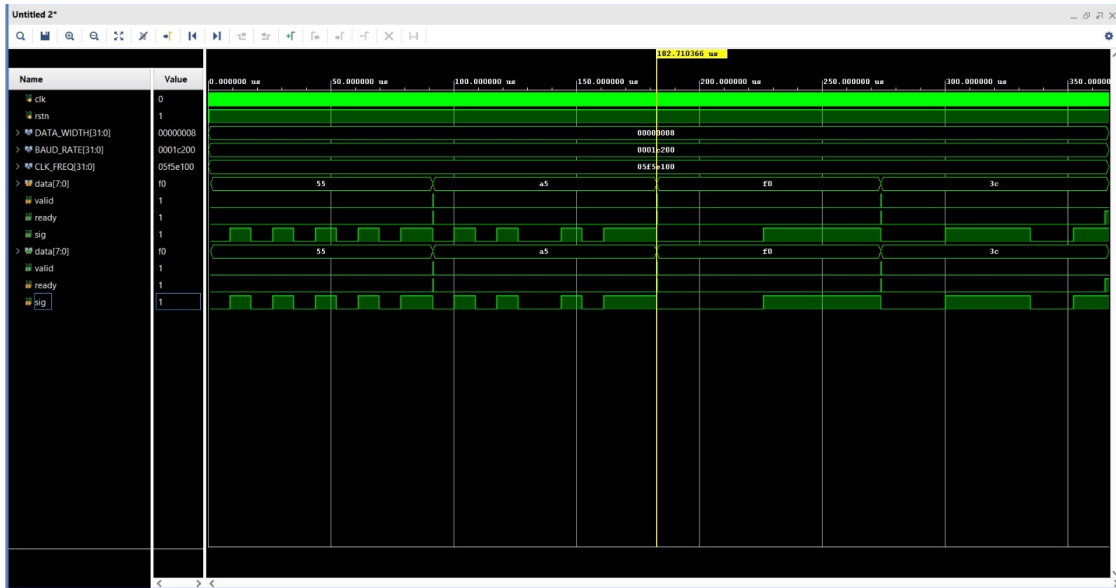


Figure 2: Vivado simulation waveform showing TX-to-RX loopback for bytes 55, A5, F0, and 3C

11.1 Reading the Waveform

- clk - toggles continuously at 100 MHz throughout the simulation
- rstn - held low briefly at the start, then goes high (1) to release reset
- DATA_WIDTH, BAUD_RATE, CLK_FREQ - the parameter values passed into the DUT (8, 115200, 100,000,000), shown here in hex (00000008, 0001c200, 05f5e100)
- data[7:0] (TX side) and valid - show each byte (55, A5, F0, 3C) being loaded into the transmitter with valid asserted
- sig (TX side) - the serialized bit stream sent out by the transmitter, toggling between high and low as each bit is shifted out at the UART baud rate
- data[7:0] (RX side) and valid - the receiver reconstructs the exact same byte (55, A5, F0, 3C) after deserializing the incoming sig signal, with valid going high once the full frame is received
- sig (RX side) - identical to the TX sig waveform, since this is a direct loopback connection

Because the RX-side data[7:0] matches the TX-side data[7:0] for every test case (55 -> 55, A5 -> A5, F0 -> F0, 3C -> 3C), the simulation confirms that the UART transmitter and receiver are functionally correct: each byte is correctly serialized, transmitted bit-by-bit, and accurately reconstructed on the receiving end. This matches the PASS results printed by the testbench's \$display statements.

12. FPGA Implementation - Basys3 Board

After successful simulation, the design was synthesized and implemented in Vivado, and the resulting bitstream was programmed onto the Digilent Basys3 board (Xilinx Artix-7 FPGA). The board was connected to a laptop via USB for both programming (JTAG) and serial communication (UART).

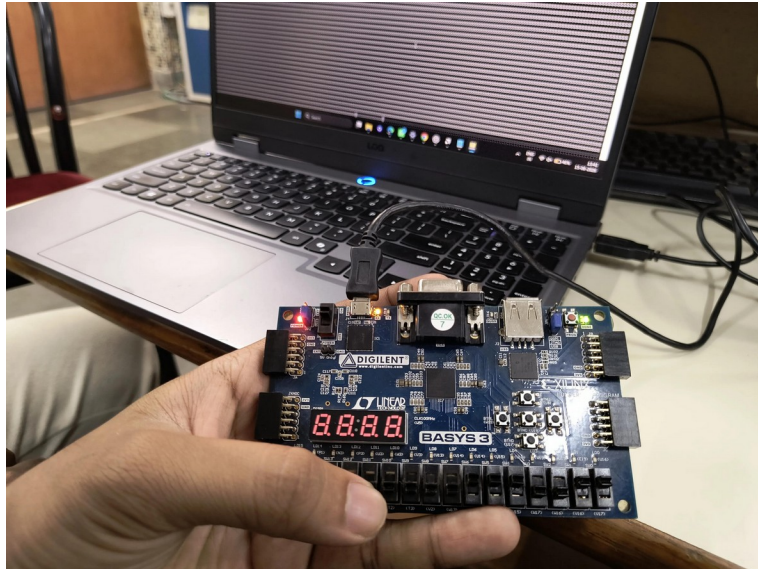


Figure 3: Basys3 FPGA board connected to PC, programmed with the UART design (Power and Done LEDs ON)

Implementation flow followed in Vivado:

- 1. Create a new RTL project and add all SystemVerilog source files (uart_if.sv, uart_tx.sv, uart_rx.sv, uart.sv, top.sv)
- 2. Set top.sv as the top module
- 3. Add the uart.xdc constraints file
- 4. Run Synthesis -> Implementation -> Generate Bitstream
- 5. Open Hardware Manager, connect to the Basys3 board, and program the device
- 6. Open a serial terminal (PuTTY/Tera Term) at 115200 baud, 8 data bits, no parity, 1 stop bit, on the COM port assigned to the Basys3
- 7. Change the switch positions on the board and observe the corresponding byte value appearing in the terminal

13. Conclusion

This project successfully demonstrates the design, simulation, and hardware implementation of a UART transmitter and receiver using SystemVerilog. By using SystemVerilog interfaces (uart_if), the design achieves a clean, modular structure where the transmitter and receiver are independently developed and verified, yet easily connected through well-defined modports.

The transmitter and receiver each follow a simple 3-state FSM (Wait -> Data -> Stop), making the control logic easy to understand and debug. The addition of a majority-vote noise filter on the receiver input improves robustness against line glitches, which is especially important for real hardware where electrical noise is present.

Functional simulation in Vivado confirmed that data loaded into the transmitter is correctly serialized at the configured baud rate (115200 bps) and accurately reconstructed by the receiver, validating the protocol timing (start bit, 8 data bits LSB-first, stop bit) and the noise-filtering logic.

On hardware, the design was successfully synthesized and programmed onto the Basys3 FPGA board. The switch inputs (SW[7:0]) were transmitted over UART and observed on a PC terminal via the USB-UART bridge, confirming that the design operates correctly in real time on actual hardware, not just in simulation.

Overall, this project reinforced key digital design concepts including finite state machines, parameter-driven (reusable) module design, SystemVerilog interfaces/modports, clock-domain timing for serial protocols, and the complete FPGA design flow from RTL to bitstream using Vivado.